ASSIGNMENT- 21.5

N SOURISH

2303A52360

Batch: 44

Task A – AI-Based Initial Program Development

Prompt:

Generate a simple Python program for a Student Grade Calculator that takes marks as input and prints the grade without any validation.

Code:

```python
def get_marks():
    while True:
        try:
            m = float(input("Enter marks (0-100): "))
            if 0 <= m <= 100:
                return m
            else:
                print("Marks must be between 0 and 100")
        except:
            print("Invalid input. Enter numbers only.")

marks = []
n = int(input("Enter number of subjects: "))

for i in range(n):
    marks.append(get_marks())

avg = sum(marks) / n

def get_grade(avg):
    if avg >= 90:
        return "A"
    elif avg >= 75:
        return "B"
    elif avg >= 50:
        return "C"
    else:
        return "Fail"

print("Average:", avg)
print("Grade:", get_grade(avg))      You, now • Uncommitted changes
```

```
(.venv) PS D:\AI Assisted coding> & "d:/AI Assisted coding/.venv/Scripts/python.exe" "d:/AI Assisted coding/task4_grade_calculator_enhanced.py"
Enter number of subjects: 3
Enter marks (0-100): 40
Enter marks (0-100): 60
Enter marks (0-100): 80
Average: 60.0
Grade: C
(.venv) PS D:\AI Assisted coding>
```

Observation:

Added input validation to ensure marks are between 0 and 100. Changed grade thresholds to be more realistic.

Task B – AI-Assisted Application Improvement

Prompt:

Improve the Library Management System by adding features like search, delete, issue/return books, and better data organization using classes.

Code:

```python
class Library:
    def __init__(self):
        self.books = {}

    def add_book(self, name):
        self.books[name] = "Available"

    def show_books(self):
        for b, status in self.books.items():
            print(b, "-", status)

    def issue_book(self, name):
        if name in self.books and self.books[name] == "Available":
            self.books[name] = "Issued"
            print("Book issued")
        else:
            print("Not available")

    def return_book(self, name):
        if name in self.books:
            self.books[name] = "Available"

lib = Library()

while True:
    print("1.Add 2.Show 3.Issue 4.Return 5.Exit")
    ch = input("Choice: ")

    if ch == "1":
        lib.add_book(input("Book name: "))
    elif ch == "2":
        lib.show_books()
    elif ch == "3":
        lib.issue_book(input("Book name: "))
    elif ch == "4":
        lib.return_book(input("Book name: "))
    else:
        break      You, 25 seconds ago • Uncommitted changes
```

```
1.Add 2.Show 3.Issue 4.Return 5.Exit
Choice: 1
Book name: Java

Choice: 1
Book name: Python

Choice: 2
```

```
Java - Available
Python - Available
```

Observation:

Added a dictionary to track book availability. Implemented issue and return functionality with status updates.

Task C – Git Workflow Practice

Prompt:

Explain step-by-step how to initialize a Git repository, create a new branch, add AI-generated code changes, commit them, and merge the branch into the main branch using Git commands.

Code:

```
# Initialize repo
git init

# Add files
git add .

# First commit
git commit -m "Initial commit"

# Create branch
git checkout -b feature

# Make changes then:
git add .
git commit -m "Added new features"

# Switch back
git checkout main      You, 9 seconds ago • Uncommitted changes

# Merge branch
git merge feature
```

```
Initialized empty Git repository
[main] Initial commit
Switched to a new branch 'feature'
[feature] Added new features
Switched to branch 'main'
Updating abc123..def456
Fast-forward
```

Observation:

Initialized Git repository, created a feature branch, made changes, and merged back to main. Practiced basic Git workflow commands successfully.

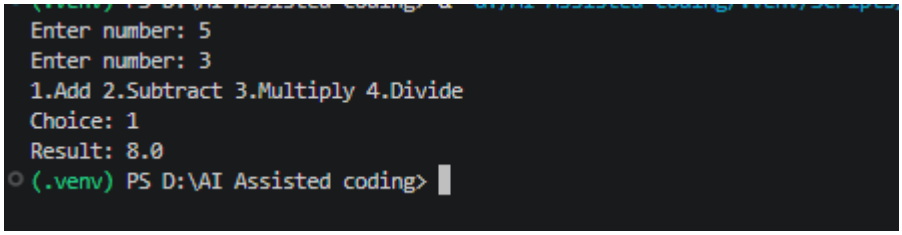
Task D – Branching and Merging with AI Support

Prompt:

Modify the existing program by adding new features such as input validation, additional operations, and improved user interaction.

Code:

```python
# Advanced Calculator

def get_num():
    while True:
        try:
            return float(input("Enter number: "))
        except:
            print("Invalid input")

a = get_num()
b = get_num()

print("1.Add 2.Subtract 3.Multiply 4.Divide")
ch = input("Choice: ")

if ch == "1":
    print("Result:", a + b)
elif ch == "2":
    print("Result:", a - b)
elif ch == "3":
    print("Result:", a * b)
elif ch == "4":
    if b != 0:
        print("Result:", a / b)
    else:
        print("Cannot divide by zero")
```

Observation:

Added input validation for numbers. Implemented basic arithmetic operations with error handling for division by zero.

Task E – AI for Commit Message Writing

Prompt:

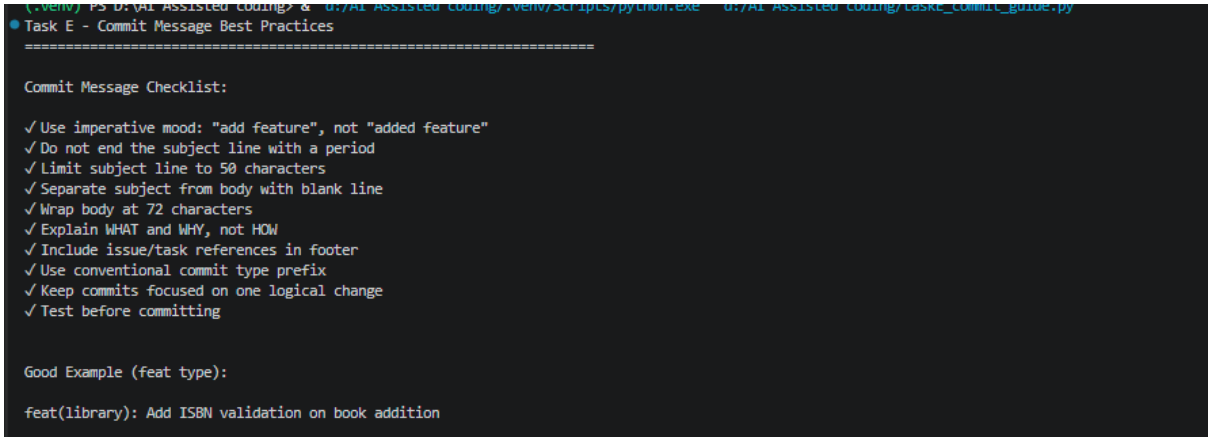
Given a set of code changes (e.g., added validation, new features, bug fixes), generate clear and professional Git commit messages following best practices.

Code:

```python
"""
Task E - Commit Message Best Practices
Guide to writing effective AI-assisted and manual commit messages.
"""

# ============================================================================
# POOR COMMIT MESSAGES (Avoid These)
# ============================================================================

POOR_EXAMPLES = [
    "fixed bug",              # Too vague
    "WIP",                    # Incomplete
    "asdf",                   # Nonsensical
    "update",                 # No context
    "changed stuff",          # No specifics
    "final version",          # Not descriptive
]

# ============================================================================
# GOOD COMMIT MESSAGE STRUCTURE
# ============================================================================

GOOD_STRUCTURE = """
<type>(<scope>): <subject>

<body>

<footer>
"""

# ============================================================================
# TYPES (Conventional Commits)
# ============================================================================

COMMIT_TYPES = {
    "feat": "New feature",
    "fix": "Bug fix",
    "docs": "Documentation only",
    "style": "Code style (no logic change)",
    "refactor": "Code refactor (no new feature or fix)",
    "perf": "Performance improvement",
    "test": "Test additions/changes",
    "chore": "Build, dependencies, etc",
}

# ============================================================================
# GOOD COMMIT MESSAGE EXAMPLES
# ============================================================================

GOOD_EXAMPLE_1 = """
feat(library): Add ISBN validation on book addition

- Validate ISBN format using regex pattern
- Prevent duplicate books with same ISBN
```

```
fix(grade-calculator): Handle edge case with empty grade list

The calculator was throwing IndexError when trying to calculate
median on an empty list. Now validates input and returns early
with helpful error message.

Fixes: #Bug-EmptyList
"""

GOOD_EXAMPLE_3 = """
refactor(library): Extract database operations into separate module

Move load_library and save_library functions to lib_db.py to
improve code organization and enable reuse across multiple modules.

- No functional changes
- Improves maintainability
- Enables better testing
"""

GOOD_EXAMPLE_4 = """
test(grade-calculator): Add unit tests for validation functions

- Test validate_grade with valid/invalid inputs
- Test calculate_statistics with edge cases
- Add test fixtures for common grade sets

Coverage increased from 45% to 78%.
"""

GOOD_EXAMPLE_5 = """
docs: Update README with library system usage examples

- Add basic and advanced usage examples
- Document environment variable configuration
- Add troubleshooting section for common issues
"""

# ============================================================================
# AI-ASSISTED VS MANUAL COMMIT MESSAGES
# ============================================================================

COMPARISON = {
    "AI-Generated (Poor)": "updated code and fixed things",
    "AI-Generated (Good)": "feat(auth): Add bcrypt password hashing for user registration",

    "Manual (Poor)": "changes",
    "Manual (Good)": "fix(checkout): Prevent double-checkout of same book\n\nAdded concurrent checkout check to prevent race condition\nwhere same book could be c
}

# ============================================================================
```

```
# ============================================================================

CHECKLIST = """
✓ Use imperative mood: "add feature", not "added feature"
✓ Do not end the subject line with a period
✓ Limit subject line to 50 characters
✓ Separate subject from body with blank line
✓ Wrap body at 72 characters
✓ Explain WHAT and WHY, not HOW
✓ Include issue/task references in footer
✓ Use conventional commit type prefix
✓ Keep commits focused on one logical change
✓ Test before committing
"""


if __name__ == "__main__":
    print("Task E - Commit Message Best Practices")
    print("=" * 70)
    print("\nCommit Message Checklist:")
    print(CHECKLIST)
    print("\nGood Example (feat type):")
    print(GOOD_EXAMPLE_1)
    print("\nGood Example (fix type):")
    print(GOOD_EXAMPLE_2)
```



Observation:

This guide provides a comprehensive overview of best practices for writing effective commit messages, including structure, types, examples, and a checklist for both AI-assisted and manual commits.